PAPERNEXUS: A Local-First Research Harness for Paper-Centered Knowledge Work

PaperNexus Project

Abstract

PAPERNEXUS is a local-first research harness for paper-centered knowledge work. It helps a researcher move from a topic or corpus to an inspectable research memory: literature candidates, legal full-text availability, parsed paper snapshots, typed knowledge-graph entities, method-evolution evidence, cross-domain research answers, retrieval benchmarks, and operational views for humans and agents. The central failure mode addressed by the system is unsupported plausibility: a research assistant may produce a convincing synthesis while losing track of which papers, claims, methods, quotes, access decisions, and evaluation outcomes justify that synthesis.

This report presents PAPERNEXUS as a functional system rather than a code artifact. As of May 10, 2026, the system has seven user-facing layers. The discovery layer turns a topic into merged paper candidates and explicit open-access status. The ingestion layer converts local PDFs or Markdown files into reusable semantic snapshots. The graph layer stores papers, problems, methods, claims, evidence, datasets, benchmarks, ideas, and method-to-method relationships. The intelligence layer answers questions through graph evidence, method lineage, cross-domain mechanism search, and idea-catalyst workflows. The evaluation layer measures retrieval and task behavior against BEIR-style, biomedical, literature-search, and custom benchmarks. The security layer supports encrypted local environment variables and sanitized release artifacts. The interface layer exposes the same research memory through a command-line interface, browser dashboard, authenticated HTTP APIs, local stdio MCP, remote streamable HTTP MCP, and Python helper scripts. The result is a research environment designed to preserve evidence, support staged reuse, and make long-running literature work auditable.

1 Introduction

Academic research increasingly depends on long chains of machine assistance: search engines retrieve candidate papers, PDF parsers extract text, language models summarize sections, graph tools connect ideas, and agents turn intermediate notes into research plans. The practical bottleneck is no longer only whether a system can find a paper. It is whether the system can preserve enough structure for a later user to ask: Where did this claim come from? Which method improved which predecessor? Was the cited paper actually available as an open PDF? Which part of the corpus supports this cross-domain analogy?

PAPERNEXUS is built around a conservative assumption:

> If research memory is represented only as files, embeddings, or prompt-time summaries, then evidence can become detached from the claims that depend on it.

The system therefore treats the research graph as the primary artifact. Papers are not only documents to retrieve; they are sources of typed objects such as problems, methods, claims, evidence, datasets,

benchmarks, takeaways, idea fragments, and future directions. Literature discovery is not only a download helper; it is a record of what was searched, merged, resolved, skipped, and legally available. Research answers are not only generated prose; they are views over an evidence-bearing graph.

This paper follows the style of functional technical reports for research harnesses such as ARIS [10]: it emphasizes the operating assumptions, architectural layers, workflow surfaces, assurance mechanisms, evaluation hooks, and deployment footprint of the system. We intentionally de-emphasize line-by-line implementation details. The goal is to explain what a researcher can do with PAPERNEXUS, how the pieces fit together, and why the system is organized around local ownership, explicit evidence, and measurable behavior.

2 Functional Thesis

The system is designed around three claims.

Research work needs durable memory. A literature review does not end when a set of snippets is retrieved. The researcher needs to revisit earlier parses, compare method families, inspect unresolved sources, and ask new questions without repeating the whole pipeline. PAPERNEXUS therefore stores raw sources, markdown cache, semantic snapshots, graph artifacts, queue state, discovery outputs, and derived overlays as local, reusable assets.

Evidence should be represented before answers are generated. For paper-centered work, the most useful automation surface is often not a chatbot response but a structured memory that can later support many responses. PAPERNEXUS places graph construction before answer construction. Question answering, method lineage, cross-domain search, and idea generation operate over committed graph state rather than relying on query-time invention.

Automation must remain inspectable. The system exposes its state through multiple surfaces: CLI commands for operators, a browser dashboard for inspection, HTTP APIs for applications, MCP tools for agents, and Python scripts for remote workflows. These interfaces are different views of the same local research memory rather than independent silos.

Evaluation and secrets must be first-class operational concerns. A research harness that discovers papers, calls external providers, and serves remote agents must be evaluated and sanitized before it is shared. PAPERNEXUS therefore includes retrieval-benchmark loaders and metrics, encrypted local environment storage, bearer-token guarded server surfaces, and a release-hygiene path that removes personal paths, accounts, local state, and generated build traces from public artifacts.

3 System Overview

Figure 1 summarizes the functional architecture. The left side introduces sources, the center maintains durable research memory, and the right side exposes human and agent workflows.

3.1 Layered Responsibilities

The separation is practical. Discovery can be repeated without rebuilding the graph. Parsing can be retried without losing the source manifest. Graph construction can be staged and committed after inspection. Interfaces can query committed state without re-running expensive upstream work.

4 Functional Workflows

4.1 Topic to Corpus

The discovery workflow starts from a topic, seed question, related-work paragraph, venue, author, dataset, or manually supplied paper. It plans complementary queries, sends them to open metadata providers, merges candidates, expands through citations when available, and resolves legal full-text sources. It uses services such as OpenAlex [7], Semantic Scholar [9], Crossref [3], arXiv [1], DBLP [4], Europe PMC [5], CORE [2], and Unpaywall [8].

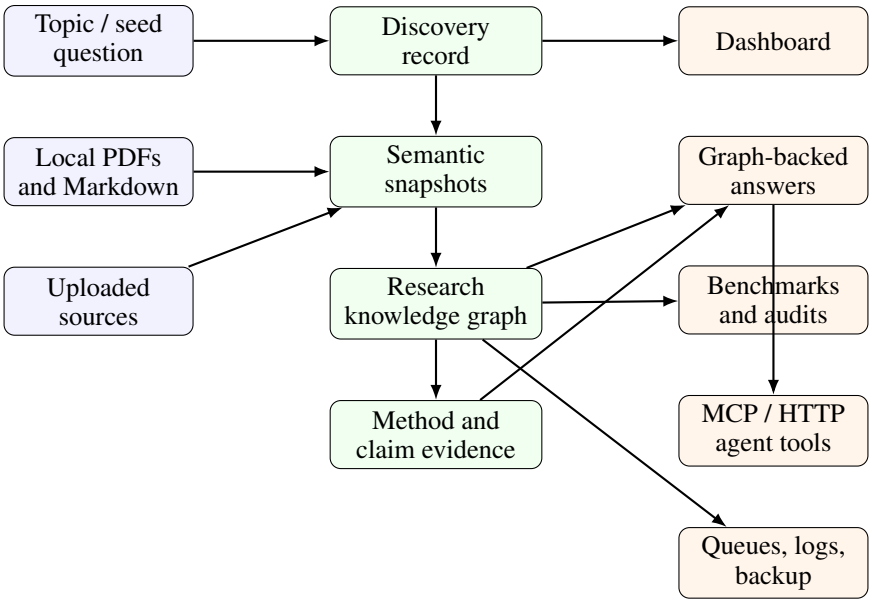

Figure 1: Functional view of PAPERNEXUS. Discovery, ingestion, graph memory, evidence, benchmark, agent, and operations layers are maintained as local artifacts or controlled interfaces without changing the local ownership model.

Table 1: Functional layers in PAPERNEXUS.

Layer	Role in the research workflow
Discovery	Turns a topic into provider queries, merged candidates, open-access status, download manifests, and importable sources.
Ingestion	Converts local PDFs or Markdown into reusable paper snapshots with parser metadata and semantic extraction results.
Graph memory	Represents papers, problems, methods, claims, evidence, datasets, benchmarks, takeaways, ideas, and relations as a persistent local graph.
Evidence	Tracks method evolution, citation contexts, exact quotes, temporal direction, confidence, and blocked or ambiguous relationships.
Interfaces	Provides CLI, Web UI, HTTP API, local MCP, remote HTTP MCP, services, logs, and Python wrappers.
Evaluation and hygiene	Measures retrieval and task behavior, stores encrypted local secrets, and sanitizes release artifacts before publication.

The important output is not merely a folder of PDFs. The workflow records what was found and why a candidate could or could not enter the corpus. A result may be a resolved open PDF, a metadata-only paper, an item requiring institutional access, an access-control-blocked page, or a URL that returned HTML instead of a PDF. This distinction matters because a research assistant should not confuse "not downloaded" with "not relevant" or with "not legally available." The current implementation persists discovery runs with JSON artifacts, Markdown reports, download manifests, import-task links, candidate provenance, and latest-run pointers.

4.2 Corpus to Research Memory

Once sources exist locally, PAPERNEXUS materializes them into reusable semantic snapshots. The default path parses PDF or Markdown, stores markdown cache and metadata, and allows later LLM-assisted enrichment to run as a separate stage. This gives the user a durable intermediate representation: if graph construction changes, the system can reuse prior parsing rather than starting from raw PDFs.

The staged pipeline is intentionally simple:

```
sources -> materialize -> llm-optimize -> build-graph
        -> merge-graph -> write-index
```

The key property is resumability. A user can run the full analysis once, continue after interruption, rebuild only dirty stages, or inspect stage outputs before committing a new graph.

4.3 Research Memory to Evidence

The graph layer makes paper-centered objects queryable. Problems, methods, claims, evidence, datasets, benchmarks, takeaways, idea fragments, and future directions become typed nodes. Relations between them become explicit edges. This enables both conventional literature exploration and higher-level reasoning tasks.

Method evolution is treated as a special evidence problem. A system should not casually claim that one method extends, improves, replaces, adapts, or uses another method. PAPERNEXUS therefore records method aliases, citation contexts, quote evidence, candidate relationships, accepted relationships, stubs, and diagnostics. Strong method-evolution relationships require support beyond name overlap: the relationship must be grounded in context, direction, and confidence.

4.4 Evidence to Answers

Research-intelligence workflows read from graph state. They can answer cross-domain questions, produce method lineages, inspect evidence for a method relationship, return a method registry, or combine these views into a unified answer. The operating invariant is that these workflows should expose existing graph evidence rather than silently generate new evidence at query time.

This gives the system a useful division of labor. Language models may help enrich snapshots or phrase responses, but the evidence substrate remains an inspectable local graph. When a researcher asks for a mechanism, lineage, or analogy, the answer can point back to the corpus state that supports it.

4.5 Research Memory to Evaluation

PAPERNEXUS now includes a retrieval-benchmark path that can evaluate the discovery and retrieval surface rather than only describe it qualitatively. The benchmark command supports closed-corpus and live-discovery modes. Closed-corpus mode follows information-retrieval semantics: the system searches a provided corpus and is scored against query relevance labels. Live mode runs the normal provider-backed discovery workflow and scores returned papers against gold identifiers or titles.

The loader surface covers custom JSON/JSONL, BEIR-style corpora and TREC qrels, LitSearch-style query/corpus exports, BioASQ question files, native TREC biomedical topics plus qrels, CSFCube faceted annotations, and flexible schemas for SAGE, ScholarQABench/OpenScholar, PaperAsk, SPARBench, and SciNetBench. The reported metrics include hit@k, exact match@k, recall@k, weighted recall@k, precision@k, F1@k, MRR@k, MAP@k, and NDCG@k. For task-style benchmarks, PAPERNEXUS can additionally score answer overlap, citation precision and recall, claim-label accuracy, relation/path overlap, and optional LLM-judged answer grounding.

5 Interface Surface

PAPERNEXUS exposes the same research memory through several interfaces (Table 2). This is a functional choice rather than a convenience layer: different users need different levels of control over the same corpus.

The MCP surface follows the Model Context Protocol [6]. Its purpose is to make research memory available to agents without asking those agents to understand the internal storage layout. Important tool families include literature discovery, research lookup, research briefing, idea catalyst, and import workflow.

Table 2: Interface surfaces and their intended users.

Interface	Typical use
CLI	Initialize a corpus, run staged analysis, inspect status, search the graph, benchmark retrieval, manage secure environment values, export backups, and manage services.
Dashboard	Inspect corpus status, graph summaries, overlays, imports, runtime configuration, and human-readable views.
HTTP API	Integrate browser and server workflows with authenticated routes for graph, method-evidence, discovery, and research-answer queries.
Local MCP	Give a local coding or research agent tool access over stdio.
Remote HTTP MCP	Expose the same agent tool surface from a running server with bearer-token authentication.
Python wrappers	Submit imports, inspect queues, query graph state, and orchestrate remote workflows from scripts.
Secure environment	Keep local provider keys and runtime secrets in an encrypted file whose encryption key is held by the system keychain.

6 Assurance Mechanisms

PAPERNEXUS does not claim that every extracted relationship is correct. Instead, it makes uncertainty and evidence boundaries visible.

Access assurance. Discovery records whether full text is open, unavailable, restricted, blocked, or malformed. This prevents downstream tools from treating access failure as absence of evidence.

Snapshot assurance. Parsing and semantic extraction are separated from graph commit. A failed parser, degenerate title, or incomplete snapshot can be detected before it pollutes the graph.

Graph assurance. The graph schema constrains the kinds of objects and relationships that can be represented. Lite projections and metadata make committed state inspectable without requiring the full graph engine for every read.

Method-evidence assurance. Strong method-evolution edges require resolved methods, citation context, quote support, confidence, temporal direction, and non-conflicting evidence. Ambiguous aliases and stubs are preserved as diagnostics rather than silently promoted.

Operational assurance. Queues, logs, backups, and staged commands allow the user to recover from interruptions and audit long-running work. This is especially important when imports, PDF parsing, and LLM enrichment run over large corpora.

Evaluation assurance. Retrieval benchmarks separate fixed-corpus scores from live-provider scores. This prevents a live API run from being mistaken for an official leaderboard run and makes provider availability, source resolution, and corpus retrieval quality visible as distinct measurements.

Credential and release assurance. Local credentials can be loaded from encrypted secure-env storage, and public release artifacts can be sanitized so that personal account names, absolute home paths, local caches, bearer-token fixtures, and LaTeX build traces do not leak into documentation or arXiv source bundles.

7 Functional Footprint

Table 3 summarizes the current feature footprint without tying the description to source files.

Table 3: Current functional footprint.

Function	Current behavior
Literature discovery	Plans queries, merges multi-provider candidates, resolves legal full text, records coverage, and bridges resolved sources into the import queue.
PDF and Markdown ingestion	Supports local corpora and uploaded import tasks with parser configuration, fallback parsing, markdown cache, and semantic snapshots.
Graph construction	Builds a typed research graph and commits a lightweight read projection for query, dashboard, and agent workflows.
Method evolution	Builds method registries, validates method-to-method evidence, records candidates and accepted relationships, and exposes lineage/evidence lookups.
Research answers	Provides graph-backed cross-domain evidence, method lineage, method evidence, method registry, and unified answer modes.
Retrieval evaluation	Runs fixed-corpus and live-discovery retrieval benchmarks over custom, BEIR-style, biomedical, literature-search, and graph-reasoning benchmark formats.
Security hygiene	Provides encrypted local secure-env storage, bearer-token authenticated server surfaces, portable path helpers, and sanitized release artifacts.
Operations	Supports services, logs, dashboard serving, authenticated remote MCP, backup export, backup unpack, and queue inspection.

8 Example Use Patterns

Literature mapping. A researcher starts with a topic such as "graph-augmented experiment planning." PAPERNEXUS searches open metadata providers, merges duplicate candidates, records which papers have legal full-text access, imports resolved PDFs, and builds a graph that supports later search over problems, methods, claims, evidence, and datasets.

Method lineage review. A researcher asks how a method family evolved. The system looks for validated method-to-method relationships, reports predecessor and successor links, and exposes the citation contexts and quote evidence that support the relationship. The user can distinguish a substantive improvement claim from a mere background comparison.

Cross-domain ideation. A researcher looks for mechanisms from one domain that might transfer to another. PAPERNEXUS uses domain bridge, analogy, catalyst, and graph-evidence workflows to propose candidates grounded in the existing corpus rather than free-form brainstorming alone.

Agent-assisted research. A coding or research agent connects to local or remote MCP. Instead of scraping folders directly, the agent can call tools for discovery status, graph lookup, research answers, paper index lookup, import workflow, and idea-catalyst operations.

Retrieval benchmarking. A researcher evaluates whether a discovery strategy is actually returning expected papers. In fixed-corpus mode, PAPERNEXUS reads a benchmark corpus, query set, and qrels, then reports ranking metrics. In live mode, the same benchmark query can exercise OpenAlex, Semantic Scholar, Crossref, arXiv, and configured optional providers. This makes external-provider behavior measurable without conflating it with closed-corpus retrieval quality.

9 Deployment Observations

The current implementation is a functional research harness with an implemented benchmark entry point, but not yet a published benchmark result suite. The observed system state supports four practical conclusions.

Table 4: Stage timing from a 10-real-paper local pipeline run. Values are means reported by the pipeline progress logs for that run.

Measured stage	Mean duration	What the measurement covers	Interpretation
Fast graph commit	0.084 s	Lightweight graph projection commit after semantic materialization.	The durable write path is small relative to parsing and LLM work.
Sync	0.59 s	Local synchronization work around the committed research memory.	Synchronization overhead is not the limiting stage in the observed run.
LLM optimize	113.7 s	Semantic enrichment through the LLM-optimization stage.	This is the dominant cost in the LLM-enhanced pipeline.
Active sync	120.4 s	End-to-end active synchronization window observed for the same workload.	The total active window tracks LLM optimization closely, with storage overhead remaining secondary.

Table 5: Production-style import timing shape after scoped import processing was enabled. These values describe normal single-paper import tasks and are separate from the 10-paper aggregate pipeline log above.

Stage	Typical time per paper	Operational reading
PDF to Markdown	0.3–1.5 s	Parser work is usually short for ordinary born-digital PDFs.
Materialize total	0.4–2.2 s	Source reading, PDF conversion, Markdown cache, and snapshot creation remain low seconds.
LLM Stage 2 semantic extraction	30–40 s	The import task normally waits on LLM request latency or provider throughput.
Fast commit	3.5–4 s	A visible fixed cost, but still smaller than LLM extraction for ordinary imports.
End-to-end import task	40–46 s	Normal LLM-enhanced single-paper imports are dominated by Stage 2.

First, the architecture is coherent: discovery feeds ingestion, ingestion feeds graph construction, method evidence attaches to graph memory, and intelligence workflows consume committed graph state. Second, the interface strategy is useful: the same local research memory can be operated from CLI, dashboard, HTTP, MCP, and scripts. Third, the evaluation surface is now concrete enough to support fixed-corpus and live-discovery retrieval experiments before a public benchmark table is reported. Fourth, release maturity should be judged through functional smoke tests, security scans, and unit tests: discovery should be exercised on small topics, method lineage should be queried through remote MCP, parser/runtime configuration should be checked on the target machine, and release artifacts should be scanned for private paths or credentials before distribution.

For this May 10 report build, local verification included the full Node test suite and a release-hygiene scan. The test suite passed 366 of 366 tests. The scan found no repository-text, arXiv-source, or PDF string matches for private home paths, personal account names, long bearer-token literals, OpenAI-style keys, GitHub-style tokens, or private-key blocks.

Local pipeline measurements. On May 11, 2026, we ran several local measurements to separate the cost of graph commit, synchronization, PDF parsing, and LLM-based semantic enrichment. The tests were run against local PAPERNEXUS corpora and authenticated local server surfaces. They are engineering measurements rather than a public benchmark suite, but they are useful for locating the current bottleneck and for validating the migration plan toward larger corpora.

The upload stress test used the HTTP import route with base64 file payloads, the background import worker, the MarkItDown PDF parser, four-way analysis concurrency, and `semanticExtraction=heuristic-only`. Therefore the measured path includes upload, queueing, PDF-to-Markdown conversion, materialization, the pipeline's LLM-optimization stage in heuristic-only mode, and fast graph commit, but excludes external LLM enrichment calls. The 100-PDF run completed without failed import tasks. Its additional cumulative substage timings were 0.418 seconds for Markdown reads, 0.499 seconds for Markdown parsing, and 1.665 seconds for semantic snapshots. Because parsing ran concurrently, 113.589 seconds of cumulative materialization work completed in 30.289 seconds of wall time, or roughly 3.3 uploaded PDFs per second for this small-PDF workload.

Table 6: Queue recovery and continue validation from a production-style import incident.

Measurement	Before recovery	Recovery evidence	Final active queue state
Historical queue audit	117 completed tasks and 33 failed tasks.	All 33 uploaded source files still existed; 24 were PDF uploads and 9 were Markdown uploads; all 33 had equivalent completed imports later in the queue.	117 completed, 0 failed, 0 pending, 0 running, and overall progress at 100%.

Table 7: Single-prompt multi-paper LLM extraction test using local `cliproxyapi` with `qwen3.6-plus` and thinking disabled.

Papers in one prompt	Completion time	Result	Interpretation
1	4.992 s	HTTP 200 with parseable structured output.	Establishes a local CPA/Qwen baseline for one-paper extraction.
10	45.561 s	HTTP 200 with parseable structured output.	Small multi-paper batching is feasible and reduces orchestration overhead.
100	256.409 s	HTTP 200, but the response was truncated at 60,848 characters and JSON parsing failed.	A 100-paper single prompt is not production-safe; batching must be bounded by prompt/output budget and recoverable by split retry.

Taken together, these measurements show that the current graph write and synchronization path is not the observed bottleneck. Without external LLM enrichment, PDF-to-Markdown conversion dominates the import path. With LLM enrichment enabled, the LLM-optimization stage dominates by one to two orders of magnitude over graph commit, depending on whether the measurement is a single import task or an aggregate pipeline run. The queue-recovery audit also validates the intended continue behavior: stale historical failures can be reconciled without deleting task logs or uploaded sources, while the active queue presents the recovered state to agents. The CPA/Qwen batch test shows why the system should not optimize by sending arbitrarily large paper batches to a single prompt: 10 papers worked, while 100 papers returned an HTTP success with unusable truncated JSON. The safer production design is prompt-budget batching, deterministic schema validation, batch-level checkpoints, and split retry/continue for malformed or oversized outputs.

This conclusion is consistent with the staged extraction design studied from arXiv:2604.28158. That system does not rely on one giant prompt over full papers. It parses PDFs into structured sections using Nougat for born-digital PDFs and GROBID as a fallback, then runs a two-phase LLM extraction process over selected evidence. The first phase uses Introduction and Method sections plus reference titles and abstracts, averaging about 21.2 references and 5.8k input / 2.1k output tokens per prompt. The second phase only processes non-background candidates, batches about 10 candidates at a time, averages about 7.9k input / 2.8k output tokens, and repairs malformed batches once before dropping persistent failures. For PAPERNEXUS, the practical implication is to keep full-text parsing local and reusable, feed the LLM only the sections needed for the target extraction, and make every LLM batch independently resumable.

10 Limitations and Future Work

PAPERNEXUS has several current limitations.

First, metadata-only discovery candidates are recorded but are not automatically promoted into full graph paper nodes unless they pass through import and materialization. Second, discovery source coverage remains incomplete: OpenReview, Papers with Code, DataCite, OpenAIRE, BASE, HAL, Zenodo, bioRxiv, medRxiv, SSRN, OSF, and Chinese scholarship providers are natural extensions. Third, systematic-review-style screening is not yet a standalone workflow with include/exclude tables and reviewer decisions. Fourth, version relations such as preprint-of, extended-by, artifact-for, and dataset-for are not yet fully represented as graph relations. Fifth, the benchmark layer has loaders and metrics but still needs reported runs on public and private gold sets. Sixth, method-evolution

Table 8: Authenticated PDF upload and import stress measurements. Each run used a fresh temporary corpus with one seed Markdown source, so committed paper counts are one larger than the number of uploaded PDFs.

Uploaded PDFs	Request payload	Enqueue latency	End-to-end time	Materialize / PDF parse	Committed graph
1	182,972 bytes	14 ms	2.026 s	1.443 s / 1.417 s	2 papers, 25 nodes, 55 edges
100	18,292,646 bytes	60 ms	30.289 s	113.589 s / 110.907 s	101 papers, 223 nodes, 1,837 edges

quality depends on extraction quality; conservative validation reduces false positives but may miss valid but weakly expressed relationships.

Future evaluation should therefore focus on functional outcomes: discovery recall against systematic-review gold sets, open-access status accuracy, parser success and metadata accuracy, graph entity and relation precision, method-edge precision, fixed-corpus retrieval quality, live-provider retrieval quality, and answer faithfulness. These measurements should complement engineering checks such as latency, throughput, queue recovery, service stability, and secure release hygiene.

11 Conclusion

PAPERNEXUS is a local-first research harness for turning paper-centered work into durable, inspectable research memory. Its main contribution is not a single retrieval model or a single agent prompt. It is the integration of discovery, ingestion, graph construction, method-evidence validation, graph-backed research intelligence, and multi-surface operation into one local workflow.

The system is designed for researchers who want automation without giving up provenance. It records what was searched, what was resolved, what was parsed, what entered the graph, what evidence supports a relationship, and what interface exposed the result. In that sense, PAPERNEXUS treats research assistance as a memory and evidence problem first, and a generation problem second.

Acknowledgments

This report was generated from the local PAPERNEXUS working tree and project documentation on May 10, 2026. The writing style was revised to follow the functional technical-report pattern exemplified by ARIS [10]; no text from that paper is reused.

References

[1] arXiv. arxiv api user manual. https://info.arxiv.org/help/api/, 2026.

[2] CORE. Core api documentation. https://core.ac.uk/services/api, 2026.

[3] Crossref. Crossref rest api. https://api.crossref.org/, 2026.

[4] DBLP. Dblp api documentation. https://dblp.org/faq/How+to+use+the+dblp+search+API.html, 2026.

[5] Europe PMC. Europe pmc restful web service. https://europepmc.org/RestfulWebService, 2026.

[6] Model Context Protocol. Model context protocol. https://modelcontextprotocol.io/, 2026.

[7] OpenAlex. Openalex documentation. https://docs.openalex.org/, 2026.

[8] OurResearch. Unpaywall api. https://unpaywall.org/products/api, 2026.

[9] Semantic Scholar. Semantic scholar academic graph api. https://api.semanticscholar.org/api-docs/, 2026.

[10] Ruofeng Yang, Yongcan Li, and Shuai Li. Aris: Autonomous research via adversarial multi-agent collaboration. https://arxiv.org/abs/2605.03042, 2026.

A Functional Command Sketch

The most common operator path is:

```
papernexus init
papernexus analyze --force
papernexus serve
```

The staged path exposes intermediate state:

```
papernexus materialize --continue
papernexus llm-optimize --continue
papernexus build-graph --continue
papernexus merge-graph --continue
papernexus write-index --continue
```

Retrieval evaluation can be run in fixed-corpus mode:

```
papernexus benchmark-retrieval ./benchmarks/scifact \
  --format beir \
  --evaluation-mode fixed-corpus \
  --k 1,5,10,20
```

Local secrets can be stored outside plain configuration files:

```
papernexus secure-env set OPENAI_API_KEY
papernexus secure-env list
```

B Remote Agent Pattern

Remote agent access is enabled by running the dashboard server with authenticated streamable HTTP MCP. A client can then call graph lookup, literature discovery, import workflow, and idea-catalyst tools against the same corpus that the local dashboard displays. This pattern is useful when the research corpus stays on one machine but agents or scripts run from another environment.